INDEPENDENT REPOSITORY ANALYSIS

Asymptotic: Beyond the Existing Reviews

Certificate soundness, adaptive precision, request semantics, and a sharper boundary algorithm

Prepared for Vladimir Reshetnikov

9 September 2026

Repository VladimirReshetnikov/Asymptotic

Pinned snapshot `921387e5ba1239bfda96e63e64e89bf63d9c41e6`

Declared package AsymptoticInverse 1.8.0; Wolfram Language 15.0+

Principal result. The strongest additional concern is a certificate-domain check that can use ambient assumptions to discharge a predicate which should instead be proved over a whole verification interval. A second finding identifies an arithmetic-precision dead end in pure-relative certification. The report also supplies a one-pass, cancellation-aware repair for an already documented nonlinear logarithmic-tail defect, rather than counting that defect again.

Evidence boundary. Source inspection and independent exact-arithmetic checks were completed. The connected Wolfram evaluator returned service errors; no native execution is claimed. Experimental patches, nine desired-contract tests, observation scripts, and the independent Python evidence accompany this article. None of the patches is represented as release-validated.

Contents

1 Executive assessment

The package's central value is not that Wolfram Language lacks asymptotics. It is the package's explicit algebra of generalized expansions, specialized inverse normal forms, and retained information about scales, branches, and errors. Native `Series`, `InverseSeries`, `Asymptotic`, and `AsymptoticSolve` remain substantial baselines. The appropriate relationship is a specialized layer over native symbolic computation, not a blanket replacement.[20, 22, 24, 25, 1]

The recommendation from this incremental review is to harden the boundary between *symbolic context* and *proof evidence*. A source condition which is valid eventually on a branch is not automatically valid on a requested finite interval. A mathematically valid root enclosure is not necessarily accurate enough for the requested tolerance. A numerical seed, an arithmetic working precision, a Taylor enclosure order, and a symbolic expansion cutoff are four different controls. The new findings expose concrete places where these distinctions matter.

1.1 Findings and material extensions

ID	Priority	Result	Evidence class
N01	High	Ambient assumptions can invalidate the certificate's claimed whole-interval source-domain verification.	Source trace plus documented native semantics; public reproducer unrun.
N02	Medium	A successful but insufficiently accurate certificate can repeatedly refine its center without raising enclosure precision. Pure-relative requests expose a fixed arithmetic floor.	Source proof and exact rational arithmetic oracle; public run pending.
N03	Low/medium	WorkingPrecision is documented as seed-only but determines automatic enclosure order, even with an explicit center; a large valid value can cause an unrelated order-validation failure.	Direct option/control-flow deduction.
N04	Medium UX	Simultaneous explicit cutoff and SeriesTermGoal have inconsistent effective precedence across the ordinary forward and Dirichlet special-function routes.	Source comparison; no coefficient error alleged.
S01	High repair	Improve the proposed conservative repair for known C04 with a closed-boundary convolution that measures the actual discarded coefficient and preserves sequential callbacks.	Mathematical proof, experimental WL patch, independent polynomial checks.

N01 is a *material sharpening* of the registered ambient-assumptions issue C05, not a claim that ambient assumptions were previously unnoticed. S01 is explicitly an improvement to the already registered C04, not a new defect count. N02–N04 are narrower additional findings relative to the maintained review register. The article avoids re-presenting the dense native-series allocation bug and the fractional-power pure-remainder bug as current discoveries; the register marks their targeted fixes verified at the reviewed snapshot.[15, 16, 18]

1.2 What the supplied artifacts establish

The archive includes the source of this article, its compiled PDF, two kinds of proposed repair, and runnable evidence. The independent Python program completed **107 checks with no failures**. These check rational rounding, polynomial boundary identities, callback ordering, and elementary tail comparisons. They neither execute Wolfram Language nor validate the full repository. The distinction is deliberately maintained throughout the report.

The certificate staging script refuses an unexpected Git blob and writes to a separate staging directory. The closed-boundary WL file is an opt-in private-function replacement for a disposable kernel. The nine `VerificationTest` cases specify desired behavior; the N04 case is a proposed policy test and is identified as such. An observation script records raw native results when a working kernel is available. No synthetic native transcript is included.

2 Scope, snapshot, and non-duplication method

2.1 The reviewed state

All repository references use the snapshot stated on the title page. The package metadata declares version 1.8.0 and a Wolfram Language 15.0+ requirement. The modular kernel and the generated standalone distribution are different delivery surfaces; changing a modular file is not the same operation as regenerating and validating the standalone.[14, 1, 15]

The review mapped the kernel directory and inspected selected implementation regions across ordinary forward/inverse jets, Fourier coefficients, explicit series operations, composite envelopes, reciprocal-log calculus, source and target coordinate adapters, special-function adapters, Dirichlet forward expansions, numerical inverse checks, refinement requests, inverse-function dispatch, and exact certificates. The coverage appendix distinguishes direct inspection from inventory-only modules. This is a substantial source audit, not an assertion that every line of all modules and tests has been formally verified.

The connector made the pinned source available as text. A local complete checkout was not obtained in this environment, and the package was not loaded locally. The Wolfram evaluator was attempted, including a minimal `Refine` semantics probe, but returned HTTP 502 service errors. Consequently, any public Wolfram result described below is an *expected result derived from source and documentation*, not an observed kernel output. Repository validation counts remain upstream reports, not new executions.

2.2 How previous work was used

The maintained `CODE_REVIEW_STATUS.md` register was read as the crosswalk for the nine review bundles and their 87 individual finding identifiers. Original review 8 was consulted for the ambient-assumptions finding; original review 2 was consulted for the nonlinear-tail analysis and its proposed conservative repair. The indexed register is the main non-duplication baseline; this report does not claim a fresh word-by-word reading of every historical article.[15, 16, 17, 18]

Entry	Closest existing item	Increment supplied here
N01	C05; review 8 F01	Moves from omitted assumption provenance to a specific false universal domain claim in an interval certificate; supplies a conditional-identity counterexample and a minimal isolation repair.
N02	D04 is nearby, but different	Concerns adaptive arithmetic after a root proof succeeds, not the rejected Automatic interval default. Establishes an exact non-dyadic rounding floor and a successful higher-precision control.
N03	D04 and D01 are nearby	Identifies an explicit-center request that fails because a supposedly seed-only option is also used as a Taylor enclosure order.
N04	D01	Identifies a concrete cross-backend conflict between two public request selectors, rather than repeating the general need to document scale-dependent cutoff units.

Entry	Closest existing item	Increment supplied here
S01	C04; review 2 F01	Gives an actual boundary algorithm, a proof of its error contract, a one-pass callback constraint, cancellation tests, and opt-in implementation code. Review 2 already suggested boundary convolution as a later sharpening.
O01/O02	X04 and X05	Supplies specific Zeta derivative-tail and Lerch derivative-closure formulas, with admission conditions and an implementation path using existing metadata, rather than repeating a generic request for more certificates.

The register does not make every unlisted concern automatically novel. Novelty here means a new concrete mechanism, counterexample, or implemented mathematical refinement relative to the checked crosswalk and the relevant original analyses. Broad recommendations such as general integration, complex sectors, a universal coefficient algebra, a stronger release gate, and a unified resource budget already exist there and are not counted again.

3 Architecture and the native Wolfram Language baseline

3.1 The representation is the main product

An ordinary precision-tracked expansion has the conceptual form

$$F(w) = b(w) + a(w)\left(\sum_{\beta<h} w^{\beta} P_{\beta}(\log w) + \mathcal{O}(w^{P}\mathcal{M}(w)^{D})\right), \qquad \mathcal{M}(w) = 1 + |\log w|, \quad w \to 0^{+}. \tag{1}$$

Here the carrier and offset are exact, while the jet and its remainder have a recorded coordinate. This representation explains why apparently elementary operations can need domain checks, relative precision, or a different scale. The explicit series and composite-envelope implementations make these choices visible rather than silently interpreting every object as a native power series.[3, 4]

Ordinary inverse coefficients are organized by exact source weights and multi-indices. Fourier-log coefficients replace a polynomial by a finite sum of polynomial amplitudes times logarithmic-phase modes. Reciprocal-log calculus instead retains a power carrier and an analytic unit in a reciprocal logarithm. These are specialized algebras with different admissibility conditions, not interchangeable views of an unrestricted transseries field.[1, 6, 7]

Target-log and source-exponential/logarithmic charts transport a problem to an existing inverse engine and reconstruct the original observable. Special-function adapters additionally retain a relationship between a finite model, the original function, and its remainder information. This makes branch and coordinate metadata operational data, not merely labels on a displayed approximation.[8, 9, 10]

3.2 A decision-oriented comparison

Need	Native starting point	When this package adds value
A local explicit expansion	Series; Asymptotic. Native Series includes fractional, negative, and logarithmic examples.	Exact sparse-weight blocks, an explicit exclusive cutoff, stored coordinates, and reusable error data.

Need	Native starting point	When this package adds value
Formal reversion of a native series	InverseSeries, followed by ComposeSeries for checks.	A supported generalized inverse normal form, logarithmic coefficient blocks, source-side selection, or multi-index coefficients.
An implicit inverse or a system	AsymptoticSolve. It admits equations, systems, parameter conditions, and real/complex settings.	A scalar specialized workflow with a persistent inverse object and selected exact cores. Native systems remain a broader interface.
A Lambert-type dominant balance	ProductLog and native symbolic transformations.	Keeping the exact core intact while expanding subordinate corrections, instead of expanding every logarithm prematurely.
Composition or differentiation	Native series algebra and ComposeSeries for compatible native data.	Explicit transportation of an admitted generalized error contract, or a meaningful refusal when its derivative is unproved.
Special-function forward asymptotics	Native Series/Asymptotic.	Adapters that retain an exact carrier, original-function provenance, fixed-parameter conditions, and family-specific bounds.
Zeta at large positive argument	Test native Asymptotic on the exact request.	A deliberate Dirichlet coordinate, explicit first omitted integer, and a stored positive-tail bound. No native failure is assumed.
Integrals or differential equations	AsymptoticIntegrate; AsymptoticDSolveValue.	These native interfaces cover problem classes beyond the scalar inverse focus of this audit.
A numerical inverse value	Numerical solving; Interval is a relevant arithmetic primitive.	The package's narrow rational enclosure procedure adds a stated local existence/uniqueness argument, provided its proof predicates are sound.

The native column summarizes documented interfaces, not benchmark outcomes.[20, 22, 23, 24, 25, 27, 28, 29, 30]

Native `Asymptotic` can infer nonstandard scales; native `AsymptoticSolve` is a much fairer comparator for implicit inversion than `InverseSeries` alone. Its order parameter should not be equated mechanically with a package exponent cutoff. Conversely, an explicit package representation is useful even when native tools can produce the same finite formula: the representation can preserve the branch, observable, uncertainty, and refinement source.[24, 25, 21, 13]

3.3 How to compare outcomes without a misleading benchmark

The supplied comparison harness records native and package outputs but does not designate a winner. A fair test fixes the source endpoint, target approach, branch, parameter assumptions, observable, and acceptable error class *before* comparing elapsed time. The number of printed terms is insufficient: a complete polynomial in a logarithm may be one coefficient block, and an exact Lambert carrier may represent information that a fully expanded expression spreads over many terms.

For the regular example $y = x + x^2$, compare the package inverse with native formal reversion after translating the order conventions. For $y = x + x^{\sqrt{2}}$, compare the selected positive germ and its residual order, not whether a display has the same syntax. For a logarithmic forward

expansion, compare the remainder divided by the proposed envelope. For Zeta, compare the first omitted integer and actual tail bound, not a generic order number. Native `AsymptoticLess` is one possible independent check on a claimed strict scale relation, but its result is itself a native symbolic computation rather than a standalone proof certificate.[26]

No runtime or memory superiority is claimed in this report. A transport failure from the connected evaluator is not evidence that a native asymptotic algorithm timed out. The comparison code is a reproducible experiment specification, not measured evidence.

4 The proof obligations used in this audit

There are three distinct meanings of an assumption in this system. A *parameter assumption* fixes the real parameter regime. An *eventual source condition* holds on some sufficiently small one-sided neighborhood of the source endpoint. A *verification-interval predicate* must hold at every point of a particular closed rational interval. A constructor can legitimately accept a condition such as $x > 1$ for an approach to $+\infty$; a later certificate over $[1/4, 3/4]$ must still reject it.

The certificate's intended mathematical implication is

$$\begin{aligned} &[\forall x \in I:\ C(x)] \ \wedge\ [\forall x \in I:\ |f'(x)| \geq \mu > 0] \\ \wedge\ &[\text{existence established in } I] \quad \Longrightarrow \quad |x_* - c| \leq \frac{|f(c) - y|}{\mu}. \end{aligned} \tag{2}$$

Replacing the first bracket by "C was assumed while simplifying" changes the theorem. Likewise, improving c without making the enclosure of $f(c) - y$ sufficiently narrow can leave the right-hand side too large forever.

The analogous obligation for jet calculus is to bound *both* uncertainty already present in the argument and finite terms discarded by the current nonlinear operation. They are logically independent sources of error. S01 develops this obligation into a usable boundary algorithm.

5 N01: Ambient assumptions can escape into a domain certificate

Classification: high-priority correctness; sharpening of C05.

5.1 Source mechanism

In `InverseCertificates.wl`, `certAttempt` obtains the retained source condition and asks `certPositiveCondition` to verify it over the complete closed interval. The latter first simplifies the condition using the positional form

```
Refine[condition, Element[x, Reals]]
```

and accepts a result of `True`. The public certificate definition does not isolate `$Assumptions`.[2]

Wolfram documents that `Refine[expr, assum]` appends the default `Assumptions` option, whose value is `$Assumptions`; an explicit `Assumptions -> ...` option instead overrides that default. Thus the positional realness assumption does not prevent ambient assumptions about x from being used.[19]

This differs materially from the earlier observation that a result can be simplified under an assumption missing from its metadata. Here an ambient condition can be mistaken for verification of that very condition on an unrelated finite interval. The register's C05 and review 8's F01 supply

the earlier provenance context; the certificate-specific consequence and counterexample below are additional.[15, 17, 16]

5.2 A counterexample requiring no special-function numerics

Construct the inverse object under neutral ambient assumptions:

```
s = Block[{$Assumptions = True},
  AsymptoticInverse[
    ConditionalExpression[x, x > 1],
    {x, Infinity}, {y, 2}]
];
```

The requested source condition is valid eventually as $x \to +\infty$. The inverse-function public wrapper retains it as a source condition while the stored explicit function is $f(x) = x$.[12]

Now compare two certificate requests for the target $1/2$, with interval $I = [1/4, 3/4]$ and center $c = 1/2$:

```
InverseCertificate[s, 1/2,
  "Interval" -> {1/4, 3/4}, "Center" -> 1/2,
  "MaxRefinements" -> 0]

Block[{$Assumptions = x > 1},
  InverseCertificate[s, 1/2,
    "Interval" -> {1/4, 3/4}, "Center" -> 1/2,
    "MaxRefinements" -> 0]
]
```

These are *unrun native reproducers*. The source predicts rejection in the neutral case and a successful certificate in the ambient case. The arithmetic is exact and elementary: the derivative is one, the residual at the center is zero, and all numbers are dyadic rationals. There is no transcendental numerical ambiguity to resolve.

Proposition 5.1. *The successful ambient-case result predicted by the inspected control flow does not establish the certificate's stated source-domain claim.*

Proof. Every $x \in [1/4, 3/4]$ satisfies $x < 1$; hence no point of the interval belongs to the retained domain $x > 1$. The equation $x = 1/2$ has its unrestricted root at $1/2$, outside that domain. Under the ambient assumption $x > 1$, however, the preliminary positional `Refine` call can return `True` without using the interval endpoints. Once that check passes, the zero residual and unit derivative satisfy the remaining containment steps. The emitted claim that the retained condition has been verified over the interval is therefore false. □

The existence of an unrestricted arithmetic root does not rescue the claim. The certificate explicitly says that its root lies within every retained source-domain condition. This is why the issue is more serious than a missing metadata field or an over-permissive display evaluator.

5.3 Minimal and architectural repairs

The narrow repair replaces the positional assumption with an explicit option:

```
Refine[condition, Assumptions -> Element[x, Reals]]
```

The staged patch implements this change. It makes the interval-domain simplification independent of ambient `$Assumptions` while preserving the structural interval checks that follow.

A stronger architecture treats certification as an isolated proof boundary. Run all proof-related symbolic transformations under a neutral ambient environment and pass only approved, recorded assumptions explicitly. Do not simply append every stored or ambient predicate to a simplifier: a condition to be established cannot be admitted as its own premise. Parameter facts and the interval predicate should remain separately typed inputs to that boundary.

This repair cannot retrospectively recover branch or parameter information already lost when an old object was constructed under hidden assumptions. That is the existing C05 provenance problem and remains distinct. The present patch prevents the specific fresh domain-check bypass; it is not a complete assumption-policy implementation for every constructor.

5.4 Acceptance criteria

The conditional-identity example must reject in both ambient environments. A matching positive control over $[2, 3]$, target $5/2$, must remain certifiable. Repeat with a contradictory ambient predicate, an ambient predicate involving an unrelated symbol, and explicit parameter assumptions. The invariant is not identical pretty-printing of all messages; it is that a whole-interval proof cannot become stronger merely because the caller temporarily assumed its conclusion.

6 N02: Successful certification can stall at fixed arithmetic precision

Classification: medium-priority convergence and performance defect.

6.1 The control-flow distinction

The automatic enclosure order is selected using `WorkingPrecision` and an optional absolute `TargetError`; the relative tolerance is not included in that initialization. Each attempt uses $B = 4(n + 10)$ significant dyadic bits when the enclosure order is n. If an attempt establishes a root certificate but misses the requested accuracy and the center was not fixed, the next iteration contracts the interval and changes the center. In this successful-but-insufficient branch the enclosure order is not increased. The other retry branch doubles it.[2]

The result is an asymmetric scheduler: failure to prove a root can trigger more accurate arithmetic, while proof of a root at inadequate accuracy can lock the arithmetic precision. Increasing the precision of the numerical seed does not repair a fixed-width interval evaluation of a non-dyadic constant.

6.2 An exact rounding floor

For a nonzero rational q, the implementation uses a dyadic grid with spacing

$$\Delta(q, B) = 2^{\lfloor \log_2 |q| \rfloor - B + 1}. \tag{3}$$

Consider $q = 1/3$. Its exponent is -2, so

$$\Delta(1/3, B) = 2^{-B-1}.$$

Since $1/3$ is not dyadic, its outward lower and upper rounded endpoints are adjacent grid points, separated by exactly this spacing.

Lemma 6.1. *If the interval evaluation of $c - 1/3$ first encloses the constant $-1/3$ by these two endpoints and then adds an exact point c with outward rounding, the resulting interval has width at least 2^{-B-1}. Its maximum absolute endpoint is therefore at least 2^{-B-2}, for every choice of c.*

Proof. Adding an exact point translates an interval without changing its width. Subsequent outward rounding cannot reduce the interval. Every interval of width d has an endpoint of absolute value at least $d/2$. Substituting $d = 2^{-B-1}$ proves the assertion. □

This is the relevant arithmetic path for the symbolic residual $x - 1/3$. It does not matter how many additional coefficients an asymptotic expansion of the identity function provides: its inverse is already exact. Nor does a better decimal seed eliminate the uncertainty introduced when the fixed-precision interval arithmetic encloses the rational constant.

6.3 A public reproducer and its interpretation

```
s = AsymptoticInverse[x, {x, 0}, {y, 2}];
InverseCertificate[s, 1/3,
  "Interval" -> {1/4, 1/2},
  "RelativeError" -> 10^-1000,
  "RefineExpansion" -> False,
  "MaxRefinements" -> 6]
```

This request is native-unrun. At the defaults, the inspected initialization gives $n = 60$, hence $B = 280$. The identity derivative is exactly one, so the residual floor becomes a floor on `CertifiedErrorBound`. A relative tolerance of 10^{-1000} at a root of magnitude $1/3$ is vastly below it. The retry loop may produce valid enclosures, but repeatedly following its unchanged-precision branch cannot meet that tolerance.

The independent Python oracle does *not* run the complete Wolfram retry loop. It checks the exact rational rounding and the unavoidable residual width. The following values are the logarithms of residual bounds at the midpoint of the corresponding rounded $1/3$ enclosure, not measured package outputs.

Enclosure order	Dyadic bits	$\log_{10}$ residual bound	Meets 10^{-1000} relative goal?
60	280	−84.8905	No
120	520	−157.1377	No
240	1000	−301.6321	No
480	1960	−590.6209	No
960	3880	−1168.5984	Yes
1920	7720	−2324.5536	Yes

The higher-order rows demonstrate that the obstruction is an avoidable arithmetic floor, not lack of an inverse or an unattainable mathematical accuracy. They do not establish universal adequacy of order 960 for other functions, intervals, or condition numbers.

6.4 Repair and scheduling policy

A small repair raises the enclosure order in the successful-but-insufficient, movable-center branch as well. The staged experimental patch doubles it with the same existing cap. This is deliberately simple: it prioritizes progress and makes the identity counterexample resolvable without inventing a new numerical heuristic.

A more efficient production policy separates three errors: seed displacement, enclosure width, and interval dependency. If the residual interval remains wide relative to the target while center changes cease improving the bound, raise arithmetic precision. If the derivative enclosure is wide because of the size of the interval, subdivide or contract the interval. If the arithmetic is already sharp and the center is poor, improve the center. These decisions should be recorded in the iteration history so a failed tolerance request explains what limited progress.

Pure-relative requests should contribute to the initial precision estimate, but that alone is not a complete repair. Large cancellation and conditioning can require more precision than a tolerance's decimal exponent suggests. After a first enclosure separates the root from zero, the proved magnitude lower bound gives a safe sufficient absolute tolerance. Use that bound for subsequent arithmetic planning, never an unverified floating-point estimate of the root magnitude.

The repair must preserve the current fixed-center semantics. A user who explicitly asks to certify the error of a particular rational center has not authorized replacement of that center. Raising enclosure precision is compatible with that request; silently substituting a better approximation is not.

7 N03: A seed-only option also controls proof arithmetic

Classification: low-to-medium API and resource-policy defect.

The main public usage text describes `WorkingPrecision` as affecting seed selection only. The certificate initializer nevertheless makes automatic enclosure order at least `WorkingPrecision`$+10$, and rejects orders above 2000. This coupling applies even when the center is explicitly supplied and no initial numerical seed is needed.[1, 2]

A minimal predicted failure is

```
InverseCertificate[s, 1/2,
  "Interval" -> {1/4, 3/4}, "Center" -> 1/2,
  WorkingPrecision -> 2000]
```

with s the identity inverse. The documented seed role would make this precision irrelevant. The implemented automatic order is 2010, which is outside its permitted range; validation fails before the zero-residual rational certificate is attempted. This is an option/control-flow deduction, not a measured native run.

The immediate correction is truthful documentation and a precise error message explaining the automatic conversion and its ceiling. The better design has separate concepts for seed precision, arithmetic precision, and analytic enclosure order. The exact linear identity does not need thousands of exponential Taylor terms merely because a high seed precision was requested. Conversely, difficult interval arithmetic can need more precision even when the asymptotic seed is already excellent.

This item shares code with N02 but addresses a different contract: N02 is a retry-policy failure for a valid accuracy request; N03 is a mismatch between the documented purpose and operational meaning of a public option. It is not the previously registered Automatic-interval issue D04.

8 N04: Conflicting request selectors have backend-dependent precedence

Classification: medium UX; no false asymptotic coefficient claimed.

The ordinary forward constructor can truncate its retained rows using a non-Automatic `SeriesTermGoal` even when an explicit cutoff has also been supplied. In the Zeta and Lerch special-function paths, the branch choosing a row frontier instead uses the term goal only when the cutoff is Automatic. With an explicit cutoff, the supplied goal is validated but does not limit the returned rows.[1, 5]

For example, the inspected paths predict different treatment of these calls:

```
AsymptoticExpansion[x + x^2 + x^3,
  {x, 0, 4}, SeriesTermGoal -> 1]

AsymptoticExpansion[Zeta[x],
  {x, Infinity, Log[5]}, SeriesTermGoal -> 1]
```

The ordinary forward call restricts the number of retained rows. The Zeta explicit cutoff selects the four integers $1, 2, 3, 4$, because $\log n < \log 5$, regardless of the requested term goal of one. Both resulting finite formulas can have valid asymptotic remainders. The issue is silent, backend-dependent interpretation of the request, not incorrect Zeta mathematics.

There are two coherent policies. One is to reject simultaneous selectors unless an explicit combination policy is supplied. The other is to specify precisely how both are combined, distinguishing an exact/maximum block count from a minimum accuracy request. Merely saying "order" is insufficient because the ordinary inverse, factored forward, and special-family interfaces count different objects.

The safest incremental change is a shared request validator that either resolves the pair before dispatch or returns a `Failure` explaining the conflict. A result should retain the effective selection policy together with the originally requested cutoff and term goal. This is a concrete extension of the existing D01 documentation concern; it is not another request to explain normalized cutoff coordinates in general.

The ninth supplied regression expresses an intentionally proposed policy: either reject this conflicting pair or return no more than the declared one-block limit. It should not be treated as an established upstream specification until a policy is selected. That distinction prevents the test suite from accidentally codifying a reviewer's preference as a pre-existing promise.

9 S01: A one-pass, cancellation-aware repair for nonlinear boundary tails

Classification: sharper implementation of existing C04, not a new defect.

9.1 What the previous review already established

The registered C04 identifies a real defect in `unitSeriesPrecision`: when the input precision is the active boundary and the requested cutoff is higher, the function can keep the input logarithmic degree while forgetting the degree created by discarded nonlinear terms. Review 2 already gives a conservative repair and explicitly mentions exact boundary convolution as a possible sharpening. The contribution here is the precise algorithm, its proof, its treatment of stateful coefficient generators, and executable independent tests. Neither the underlying defect nor the broad idea of a boundary convolution is claimed as new.[15, 18, 1]

Let $L = \log w$, $\mathcal{M}(w) = 1 + |L|$, and suppose the argument supplied to an analytic unit operation

is

$$U(w) + E(w), \qquad U(w) = \sum_{\alpha \in A} w^{\alpha} P_{\alpha}(L), \qquad E(w) = \mathcal{O}(w^{P} \mathcal{M}(w)^{D}),$$

where A is finite, $\min A > 0$, and $P > 0$. The known jet has been normalized so that all retained weights are below its uncertainty boundary. For a requested working cutoff K, set $c = \min(P, K) > 0$.

The standard conservative fix bounds the degree of all powers which might reach c. That is safe but may be substantially weaker than necessary when different Taylor contributions cancel at exactly the boundary. A coefficient polynomial is also more informative than the maximum degrees of its summands.

9.2 Closed-boundary theorem

Theorem 9.1 (Boundary coefficient controls the discarded logarithms)**.** *Suppose $H(t) = \sum_{n \geq 0} h_n t^n$ is analytic in a neighborhood of zero, with coefficients fixed on the approach under consideration. Form the complete finite sum of terms of weight at most c in*

$$\sum_{n=1}^{\lfloor c/\alpha_0 \rfloor} h_n U(w)^n, \qquad \alpha_0 = \min A.$$

Let $Q_c(L)$ be its combined coefficient at weight c, or zero if there is none. Define

$$d_c = \max(0, \deg Q_c), \qquad D_{\mathrm{out}} = \begin{cases} \max(d_c, D), & c = P, \\ d_c, & c < P, \end{cases}$$

where the zero polynomial contributes degree zero for this bound. Then the terms of weight strictly below c, together with $H(0)$, approximate $H(U + E)$ with remainder

$$\mathcal{O}\big(w^{c} \mathcal{M}(w)^{D_{\mathrm{out}}}\big).$$

Proof. Positive valuation implies $U(w) \to 0$; the stated positive remainder precision implies $E(w) \to 0$. Boundedness of H' on a sufficiently small neighborhood gives $H(U+E) - H(U) = \mathcal{O}(E)$. Put $N = \lfloor c/\alpha_0 \rfloor$. The analytic Taylor tail is $\mathcal{O}(U^{N+1})$. Its smallest power weight is $(N+1)\alpha_0 > c$, so its polynomial logarithmic growth is absorbed by the strict power gap: it is $o(w^c)$.

Each of the finitely many retained Taylor powers is a finite power–log sum. Terms whose weights exceed c are likewise $o(w^c)$. The combined weight-c term is exactly $w^c Q_c(L)$, which has the asserted degree envelope. When $P = c$, the input uncertainty must be combined with this term; when $P > c$, it too is $o(w^c)$. These estimates prove the result. No cancellation of the unknown input error has been assumed. □

The theorem is pointwise in any fixed admitted parameters. It does not assert a uniform neighborhood or constant as parameters vary, and it does not apply to a nonanalytic outer function at zero without additional analysis. Those restrictions are important: this is a repair inside the ordinary analytic unit algebra, not a general transseries composition theorem.

9.3 Algorithm and the callback-state hazard

An ordinary open-cutoff product retains weights $\beta < c$. The proposed helper instead retains $\beta \leq c$. Repeated closed products compute the necessary powers of U; their weighted coefficients are accumulated into one jet, including the complete boundary polynomial. Only after all contributions have been merged is the boundary discarded and its degree recorded.

The original coefficient generators can have mutable recurrence state. In particular, the binomial generator updates a local coefficient by multiplication with $(r-k+1)/k$. Reusing such a generator in a separate second pass is not equivalent to starting the sequence again. A purported boundary repair that first calls the existing open-cutoff routine and then calls the same generator again can therefore create incorrect coefficients.[1]

The supplied `ClosedBoundaryPatch.wl` avoids this problem: it calls `cf[k]` exactly once for each visited positive integer k, in increasing order, and uses that value for both the retained jet and the boundary. A returned `Null` retains the existing meaning of permanent termination. A zero coefficient alone is not treated as termination, because later coefficients may be nonzero. The output is the usual triple $\{T, c, D_{\mathrm{out}}\}$, so the experimental patch changes neither the public result head nor the precision representation.

```
closedPower = 1;
closedAnswer = 0;
for k = 1, 2, ...:
    closedPower = MultiplyKeepingWeightsAtMostC(closedPower, U);
    stop if closedPower is zero;
    coefficient = cf[k];                (* exactly once *)
    stop if coefficient is Null;
    closedAnswer += coefficient * closedPower;
Q = CombinedCoefficientAtWeightC(closedAnswer);
return {WeightsStrictlyBelowC(closedAnswer), c,
        Max[DegreeEnvelope(Q), inputDegreeIfBoundaryActive]};
```

The actual WL file uses the package's existing exact comparison and normalization routines. It checks candidate-pair, output-size, and Taylor-depth budgets. Those are local safeguards, not a solution to the previously registered global resource-accounting problem. A complete boundary can itself be large when many combinations have the same weight; the correct response to an exceeded budget is a failure or a separately justified coarse bound, never silently dropping boundary contributions.

9.4 Examples which distinguish the repairs

For $U = wL$, input uncertainty $\mathcal{O}(w^2)$, and a requested cutoff greater than two,

$$e^{U+\mathcal{O}(w^2)} = 1 + wL + \mathcal{O}(w^2\mathcal{M}^2), \qquad \log(1 + U + \mathcal{O}(w^2)) = wL + \mathcal{O}(w^2\mathcal{M}^2).$$

The weight-two coefficients $L^2/2$ and $-L^2/2$ cannot be bounded by a constant multiple of w^2. This is the known C04 mechanism.

For a cancellation example, let

$$U = wL - \tfrac{1}{2}w^2L^2, \qquad P = 3, \quad K = 2.$$

The boundary polynomial of e^U is

$$-\tfrac{1}{2}L^2 + \tfrac{1}{2}L^2 = 0.$$

Thus $e^{U+\mathcal{O}(w^3)} = 1 + wL + \mathcal{O}(w^2)$ is a valid, sharper statement. A rule using only the largest degree in U cannot see this cancellation. It is unnecessary to claim exact termination: the next nonzero terms still exist, but are $o(w^2)$.

The independent Python oracle compares closed-boundary accumulation with explicitly expanded polynomials on a collection of small rational-weight examples. It also checks callback order and cancellation. This provides useful evidence for the mathematics and proposed algorithm. It does not test Wolfram evaluation order, package dispatch, or every caller of `pUnitSeries`; the native regression suite remains an essential acceptance step.

9.5 Integration requirements

Before adopting the patch, verify every caller's preconditions, especially positive input valuation and analyticity at the unit center. Test direct power/log/exp operations, general analytic observables, forward expansion, inverse-function composition, and source-coordinate reconstruction. Compare all retained coefficients before and after the patch; for affected inputs, the change should be in a justified remainder degree rather than an unexplained coefficient change.

The replacement is intentionally opt-in and uses a private symbol. Private implementation names are not a stable compatibility contract. Integrating the algorithm into the canonical source, regenerating the standalone, and running the full native suite are separate tasks, none of which was performed in this environment.

10 Additional audit results and rejected allegations

10.1 The certificate's mathematical core should be retained

Once continuity, a whole-interval source predicate, and a nonvanishing derivative have genuinely been established, the residual-containment and endpoint-sign routes have sound mathematical motivations. The implementation then intersects a symmetric residual bracket with a signed derivative correction. It explicitly distinguishes the stored explicit function from an unspecified input-remainder family and avoids asserting a global inverse-branch certificate. N01 attacks the discharge of one premise; it does not show that the mean-value theorem or the rational enclosure formulas are wrong.[2]

Likewise, N02 is not evidence of a falsely certified accuracy. The existing comparison with the requested tolerance is the safeguard that makes the controller fail rather than silently return an under-accurate answer. The improvement is to make the controller progress when additional arithmetic precision can meet the request.

10.2 Checks which prevented false positives

A local reading of a coordinate-specific numerical checker suggests that it does not check every retained source condition. The public `InverseNumericalCheck` wrapper, however, performs a common source-domain check after the coordinate-specific result is returned. That dispatch path must be included before alleging a missing-domain defect. This audit found no new public counterexample on that basis.[1, 11, 8]

Similarly, the affine Lambert-threshold adapter initially inherits inner data, but its completed result explicitly replaces the target limit by the transformed threshold offset $-\,\mathrm{scale}/e$. The final assembly, not an intermediate association, determines the public endpoint. A suspected stale-limit issue was therefore excluded.[10]

A generic refinement fallback can look as though it loses coordinate provenance. Earlier constructor-specific and conditional-source replay branches must first be checked for reachability. No new public reproduction was established for that suspicion, so it is not promoted into the findings table. The already recorded limitations of generic precision replay are not counted again.[3, 13, 15]

Finally, a value-only perturbation $\mathcal{O}(w^{\rho})$ can have a large oscillatory derivative, but using that observation to criticize the ordinary inverse remainder theorem here would ignore the documented matching derivative hypothesis. Review 2 explicitly records the value-and-first-derivative meaning of `InputRemainder`. The correct future task is making such contracts easier to supply and inspect, not pretending that no derivative condition exists.[18]

10.3 Specialized engines: useful limits rather than invented defects

The Fourier engine uses a finite algebra of logarithmic Fourier modes, checks real conjugacy, and rejects an oscillatory leading core without a provable eventual sign. Its error envelopes avoid using the zeros of a trigonometric coefficient as a denominator or error scale. A hard frequency budget is different from silently truncating the spectrum. No additional incorrect coefficient was established for this engine in the present audit.[6]

The reciprocal-log operations carry an analytic-unit contract and distinguish admitted exact implicit models from unknown omitted sectors. That is a narrower and stronger guarantee than blindly differentiating an arbitrary magnitude remainder. The restriction can be inconvenient, but it is not by itself a bug.[7]

The inspected Zeta and Lerch forward coefficient formulas and their stated value-tail constructions are mathematically well motivated. The remaining opportunities below concern making more of that already available evidence usable, not replacing correct formulas merely to produce another implementation.[5, 31, 32]

11 Concrete development opportunities beyond the general backlog

The existing register already calls for more derivative contracts, quantitative bounds, capabilities, integration, and richer scales. Repeating those ambitions would add little. This section instead develops two small, mathematically closed extensions of a backend which is already present, and gives implementation gates which connect directly to the new findings.[15, 5]

11.1 O01: derivative-tail contracts for the Zeta backend

For fixed real $S > 1$, the Zeta backend retains an initial segment of the convergent defining series and records the first omitted integer m. Its value tail satisfies

$$m^{-S} \le R_m(S) = \sum_{n=m}^{\infty} n^{-S} \le m^{-S}\left(1 + \frac{m}{S-1}\right).$$

These lower and upper bounds are already reflected in its metadata; they should not be presented as new additions of this report.[5, 31]

For an affine argument $S = ax + b$ tending to positive infinity and a fixed nonnegative integer r, termwise differentiation gives

$$\frac{d^r}{dx^r} R_m(ax+b) = (-a)^r \sum_{n=m}^{\infty} (\log n)^r n^{-S}. \tag{4}$$

The defining series and its fixed derivatives converge uniformly on each half-plane $\Re S \ge 1 + \epsilon$, justifying the operation. Equivalently, the defining-series derivative identity is recorded by DLMF.[31]

For $m \ge 2$ and $S \log m \ge r$, the function $t \mapsto (\log t)^r t^{-S}$ is nonincreasing for $t \ge m$. Integral comparison yields a usable explicit enclosure:

$$(\log m)^r m^{-S} \le T_{m,r}(S) \le (\log m)^r m^{-S} + I_{m,r}(S), \tag{5}$$

$$I_{m,r}(S) = m^{1-S} \sum_{j=0}^{r} \binom{r}{j} (\log m)^{r-j} \frac{j!}{(S-1)^{j+1}}. \tag{6}$$

To verify the formula, substitute $t = me^v$ in the integral, expand $(\log m + v)^r$, and integrate $v^j e^{-(S-1)v}$ over $[0, \infty)$. Multiplication by $|a|^r$, together with the sign $(-a)^r$, gives the derivative enclosure in (4). For $m = 1$ and $r > 0$, the first term is zero and the effective start can be moved to two; the value case $r = 0$ retains the ordinary bound.

This supplies a concrete family-specific derivative contract, rather than a user declaration with no generated justification. The finite retained derivative is trivial to compute; the key added information is the tail and the finite conditions under which it holds. The initial scope should be affine arguments and fixed derivative order. General nonlinear arguments introduce Faà di Bruno terms and merit a separate contract rather than an unnoticed extension of this one.

The derivative metadata should store the equation, derivative order, affine slope, first omitted integer, and the sufficient bound conditions. The first version can expose a dedicated derivative route without assuming that every generic coordinate solver can invert $w = e^{-S}$ into its preferred representation. A later generic adapter should consume an explicitly declared capability rather than infer it from a function name.

11.2 O02: Lerch derivatives remain in the admitted family

For fixed $|z| < 1$, real s, and positive a, the Lerch defining series gives

$$\partial_a^r \Phi(z, s, a) = (-1)^r (s)_r \Phi(z, s + r, a). \tag{7}$$

Geometric decay of $|z|^n$ dominates the polynomial factors produced by fixed derivatives, so termwise differentiation is justified locally for $a > 0$. This is a direct consequence of the defining sum, not a claim that the package already implements this derivative route.[32]

The existing coefficient family is

$$\Phi(z, s, a) \sim \sum_{k \geq 0} \frac{(-1)^k (s)_k}{k!} M_k(z) a^{-s-k}, \qquad M_0(z) = \frac{1}{1 - z}, \quad M_k(z) = \mathrm{Li}_{-k}(z) \ (k > 0).$$

Its value-tail implementation uses Taylor's theorem summed against geometric weights. The parameter shift $s \mapsto s + r$ remains inside the present fixed-real-parameter scope.[5]

Differentiating a coefficient term introduces $(-1)^r (s + k)_r$, and the polynomial identity

$$(s)_k (s + k)_r = (s)_r (s + r)_k$$

shows exact compatibility with applying the existing backend to the shifted family. Thus the proposed derivative route can reuse both coefficients and the existing quantitative value-tail construction, with appropriate index alignment and multiplication by $|(s)_r|$. For an affine argument, include the corresponding power of its slope.

Nonpositive integer s needs explicit termination handling: the Pochhammer factor can vanish, and a sufficiently high derivative is exactly zero. That is a useful regression family, not an exceptional numerical division case. The $z = 0$ family is another exact control. Negative real z can make a moment vanish without terminating all later moments; first-omitted-nonzero logic must therefore be retained. No division by a possibly zero coefficient is needed.

11.3 Use quantitative tails without conflating them with asymptotic envelopes

The Zeta and Lerch result objects already contain quantitative bounds and their conditions. Generic composite arithmetic deliberately falls back to nonnumeric asymptotic envelopes. That fallback is conservative, but loses useful evidence which is available for these particular sources.[5, 4]

A bounded extension can retain a *signed finite-range tail certificate* alongside the ordinary asymptotic descriptor. Addition and multiplication then use interval operations on actual function ranges; monotone observables can be applied to a verified interval only after establishing their domain on that interval. This is more specific than attaching a generic `Certified -> True` flag to an $\mathcal{O}$-symbol.

For example, if a positive Zeta tail is known to lie in $[A, B]$, a finite partial sum p gives a function interval $[p + A, p + B]$. A logarithm is then bounded by the logarithms of those endpoints after positivity has been checked, not merely because the asymptotic leading coefficient is positive. The logical separation from N01 remains essential: sufficient bound conditions must be verified on the finite evaluation range, not accepted as ambient assumptions.

12 Prioritized implementation and acceptance plan

12.1 First gate: protect finite certificates

Stage N01's explicit-assumptions patch and N02's arithmetic-progress patch separately enough that each can be reverted and tested. A source-domain rejection test should be invariant under unrelated ambient assumptions and under an ambient assumption equal to the very predicate being checked. Run the identity and nondyadic-target controls before involving exponential or logarithmic enclosures.

For N02, instrument each iteration with arithmetic bits, enclosure order, residual width, derivative separation, current center, certified bound, and requested sufficient tolerance. The current history already reports some of these fields; extend that record rather than inventing a hidden controller state. A successful root proof with a poor residual enclosure should trigger arithmetic work, not endlessly ask the symbolic inverse for more terms.[2]

The supplied simple doubling patch is deliberately not claimed optimal. Its purpose is to break the demonstrated fixed-precision dead end. A production controller can compare progress in center movement, interval diameter, and residual width, escalating only the component that limits the certificate. Keep the hard arithmetic ceiling and return the best valid enclosure when the accuracy goal remains unmet.

12.2 Second gate: repair nonlinear tails with differential tests

Integrate S01 into the canonical kernel after verifying its callers. Compare retained coefficients with the unpatched version on unaffected cases; compare boundary degrees with explicit small polynomial expansions on affected cases. Include exact boundary resonance, boundary cancellation, rational noninteger weights, vanishing Taylor coefficients followed by nonzero ones, and stateful binomial coefficient generators.

Test the resulting object through `SeriesLog`, `SeriesExp`, `SeriesObservable`, composition, and source-chart reconstruction. The aim is a valid remainder after a chain of operations, not only a single successful private-function test. Existing budgets must fail deterministically; a patch that fixes a logarithmic degree by allocating an uncontrolled full product is not acceptable.

12.3 Third gate: resolve request semantics before adding more adapters

Select and document the policy for simultaneous cutoff and term-goal selectors. Then apply one validator across ordinary and special routes. Avoid silently changing the meaning of a call when recognition switches from the generic engine to an optimized backend. Preserve requested and

effective values separately so a caller can tell whether the result stopped because of a cutoff, a term limit, exact termination, or input precision.

This gate is narrower than the existing general capability-matrix request: it should be possible to implement and test the four or five request states of a single selector pair without a complete object-schema redesign. Likewise, N03's documentation correction does not need to wait for a complete resource-budget redesign.

12.4 Fourth gate: add small evidence-preserving extensions

Implement O01 and O02 first as family-specific derivative operations with explicit quantitative conditions. Validate their finite formulas against symbolic differentiation and their bounds against the independent defining-series arguments above. Only then route them through generic calculus. This sequence expands useful functionality while keeping the mathematical proof obligations local and inspectable.

Complex Stokes analysis, general multivariate transseries, uniform parameter limits, and unrestricted implicit systems remain much larger projects already recognized by prior reviews. They are not presented here as fresh recommendations, nor should this focused certificate and tail work be blocked on solving them.[15]

13 Reproduction, patch status, and artifact use

13.1 What was executed

The independent oracle was executed with Python and SymPy. Its machine-readable report, `evidence/python_checks.json`, records 107 passed checks and zero failures. The tests use exact rational arithmetic where a proof step is rational; polynomial coefficient comparisons are symbolic. A finite-sum tail test is labeled as such and is not substituted for the infinite-series proof in the article.

Python syntax checks also passed for the supplied Python files. The certificate source transformation was exercised on explicit anchor fixtures, including rejection of a second application. That tests the transformation's mechanics, not application to a local checkout: no complete upstream checkout was available here. The staging program consequently retains a pinned Git-blob guard which the user must satisfy on the actual source file.

13.2 What was not executed

No native WL test, package comparison, patched kernel load, complete repository suite, standalone regeneration, or package build was executed. All supplied WL tests and experiments are marked native-unrun. This is a material limit of the evidence, not a reason to discard a direct source-level contradiction or a mathematical proof. It does mean that native integration and dispatch behavior still require confirmation.

13.3 Suggested local workflow

Use a disposable clone at the pinned snapshot and a fresh kernel. Load the modular package from its canonical kernel file; this avoids accidentally testing an old generated standalone after editing only a module. Run `native_probe.wl` on the unmodified snapshot and save the actual output with `$Version` and the commit hash.

Run the certificate staging script on the clone and inspect the generated unified diff. It does not overwrite the clone. Apply the reviewed changes only to a disposable worktree, load that version in another fresh kernel, and then load `ClosedBoundaryPatch.wl` for the experimental boundary replacement. Execute `TestReport` on `regressions.wlt`. The policy test for N04 is expected to need the selected request-policy change; it is intentionally not repaired by the two mathematical patches.

The supplied comparison harness is a bounded observation tool for built-in and package behavior. Its sample order arguments do not by themselves establish matched precision contracts; normalize the outputs as described earlier before making performance comparisons. No benchmark table in this article is populated from predictions.

13.4 Bundle contents

File	Purpose and status
`article/article.tex`, `article/article.pdf`	This article and its compiled rendering.
`code/reference_checks.py`	Executed independent mathematical checks; not a WL emulator.
`code/stage_certificate_patch.py`	Experimental pinned-blob staging of N01/N02 changes; no automatic repository overwrite.
`code/ClosedBoundaryPatch.wl`	Experimental one-pass private-function replacement for S01; native-unrun.
`code/regressions.wlt`	Nine desired-contract tests, one explicitly a request-policy proposal; native-unrun.
`code/native_probe.wl`	Raw observations for certificate and selector findings; native-unrun.
`code/native_comparison.wl`	Bounded native/package comparison experiments; native-unrun.
`evidence/python_checks.json`	Actual independent-check results and arithmetic-floor table.
`evidence/findings.csv`	Finding classification and relationship to prior reviews.
`evidence/source_coverage.json`	Inspected source regions, pin, and execution limitations.
`README.md`, `requirements.txt`	Reproduction and build instructions; Python dependency.

14 Conclusion

The most important additional result is the separation between an eventual branch assumption and a proved finite-interval predicate. At the certificate boundary, that distinction is part of correctness, not merely provenance. The precision controller exposes a different distinction: existence of a proved root and achievement of a requested error goal are separate milestones, and improving a seed cannot always compensate for a fixed enclosure grid.

The proposed nonlinear-tail algorithm illustrates how an existing review can be extended without being repeated. A complete closed-boundary coefficient yields a sharper bound, but it must be computed in a way that preserves coefficient-generator state and respects resource limits. The independent tests support that algorithm; they do not substitute for native integration tests.

The repository remains a useful specialized complement to Wolfram Language. Its exact-weight jets, retained cores, explicit uncertainty contracts, and evidence-bearing result objects are the

features to preserve. The next improvements should strengthen those contracts at their interfaces, then extend the backend in small mathematically closed steps rather than inflate a new list with already recorded issues.

A Source map and evidence granularity

The following map identifies the principal source coverage. Complete indicates that the file text was consulted, not that every path was verified. Line ranges refer to the pinned modular source, not to the generated standalone. A filename in an inventory is not counted as a complete source audit. Historical-review coverage is explicitly limited to the maintained full register and the original sections listed below.

Source	Regions	Use in this review
`AsymptoticInverse.wl` (modular)	Selected regions	Public contract, jet arithmetic, forward assembly, inverse dispatch, evidence wrappers, module loading.
`InverseCertificates.wl`	Complete	Domain proof, exact interval arithmetic, seed and accuracy controls, refinement.
`SeriesOperations.wl`	Complete	Precision transport, observables, composition, refinement reachability.
`SeriesEnvelopeArithmetic.wl`	Complete	Conservative multiscale fallback and evidence boundary.
`DirichletSpecialFunctions.wl`	Complete	Zeta/Lerch coefficients, selectors, quantitative tails.
`FourierCoefficients.wl`	Complete	Finite modes, construction, envelope and residual scope.
`ReciprocalLogOperations.wl`	Complete	Supported analytic contracts and derivative/-composition hooks.
`CoordinateInverse.wl`	Complete	Target logarithm transformations and numerical route.
`SourceCoordinates.wl`	1–165	Source reconstruction, contract transport, dispatch.
`SpecialFunctionAdapters.wl`	Complete	Erfc/Gamma/threshold models, final affine metadata, numerical checks.
`NumericalInverseChecks.wl`	Complete	Common source-domain check and observable reconstruction.
`InverseFunctionExpressions.wl`	145–end	Public condition retention and composition into enclosing jets.
`RefinementRequests.wl`	Complete	Symbolic block goals versus numerical certificate requests.
`PacletInfo.wl`	Complete	Declared version and supported kernel generation.
`CODE_REVIEW_STATUS.md`	Complete	Nine-bundle, 87-ID non-duplication baseline.
Review 8 article and findings CSV	Article 1–220; CSV complete	Prior ambient-context finding and evidence distinction.
Review 2 article	130–230	Prior nonlinear-tail repair, derivative hypothesis, native comparison.

The map is deliberately not a test-coverage claim. There are additional kernel modules, tests, documentation, historical articles, and generated artifacts whose complete behavior is not established

by the regions above. The final findings were chosen for a direct source path and an independently checkable mathematical or documented-language argument.

References

[1] Vladimir Reshetnikov, Canonical main kernel: public interfaces, jets, constructors, and evidence dispatch. `AsymptoticInverse.wl` (pinned source). Snapshot stated on the title page.

[2] Vladimir Reshetnikov, Exact interval certificates: domain predicates, rounding, and adaptive controller. `InverseCertificates.wl` (pinned source). Snapshot stated on the title page.

[3] Vladimir Reshetnikov, Explicit series operations and source/recipe refinement. `SeriesOperations.wl` (pinned source). Snapshot stated on the title page.

[4] Vladimir Reshetnikov, Composite error envelopes and conservative arithmetic. `SeriesEnvelopeArithmetic.wl` (pinned source). Snapshot stated on the title page.

[5] Vladimir Reshetnikov, Zeta and Lerch forward expansions and value bounds. `DirichletSpecialFunctions.wl` (pinned source). Snapshot stated on the title page.

[6] Vladimir Reshetnikov, Finite Fourier coefficient algebra and inverse construction. `FourierCoefficients.wl` (pinned source). Snapshot stated on the title page.

[7] Vladimir Reshetnikov, Analytic reciprocal-log composition and differentiation. `ReciprocalLogOperations.wl` (pinned source). Snapshot stated on the title page.

[8] Vladimir Reshetnikov, Logarithmic target coordinate transformations. `CoordinateInverse.wl` (pinned source). Snapshot stated on the title page.

[9] Vladimir Reshetnikov, Source coordinate reconstruction. `SourceCoordinates.wl` (pinned source). Snapshot stated on the title page.

[10] Vladimir Reshetnikov, Special inverse models and threshold adapters. `SpecialFunctionAdapters.wl` (pinned source). Snapshot stated on the title page.

[11] Vladimir Reshetnikov, Numerical inverse evidence and source-domain checking. `NumericalInverseChecks.wl` (pinned source). Snapshot stated on the title page.

[12] Vladimir Reshetnikov, Conditional source retention and inverse-function dispatch. `InverseFunctionExpressions.wl` (pinned source). Snapshot stated on the title page.

[13] Vladimir Reshetnikov, Structured refinement goals. `RefinementRequests.wl` (pinned source). Snapshot stated on the title page.

[14] Vladimir Reshetnikov, Paclet metadata. `PacletInfo.wl` (pinned source). Snapshot stated on the title page.

[15] Vladimir Reshetnikov, Maintained crosswalk of previous review findings and status. `CODE_REVIEW_STATUS.md` (pinned source). Snapshot stated on the title page.

[16] Vladimir Reshetnikov, Review 8: original article; consulted opening sections and ambient-context discussion. `asymptotic_repository_audit.tex` (pinned source). Snapshot stated on the title page.

[17] Vladimir Reshetnikov, Review 8 finding ledger, including F01. `findings.csv` (pinned source). Snapshot stated on the title page.

[18] Vladimir Reshetnikov, Review 2: nonlinear boundary precision and conservative repair; derivative contracts; native comparison. `asymptotic-review.tex` (pinned source). Snapshot stated on the title page.

[19] Wolfram Research, *Refine*, Wolfram Language documentation. `https://reference.wolfram.com/language/ref/Refine.html`. Accessed 9 September 2026.

[20] Wolfram Research, *Series*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/Series.html. Accessed 9 September 2026.

[21] Wolfram Research, *SeriesData*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/SeriesData.html. Accessed 9 September 2026.

[22] Wolfram Research, *InverseSeries*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/InverseSeries.html. Accessed 9 September 2026.

[23] Wolfram Research, *ComposeSeries*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/ComposeSeries.html. Accessed 9 September 2026.

[24] Wolfram Research, *Asymptotic*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/Asymptotic.html. Accessed 9 September 2026.

[25] Wolfram Research, *AsymptoticSolve*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/AsymptoticSolve.html. Accessed 9 September 2026.

[26] Wolfram Research, *AsymptoticLess*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/AsymptoticLess.html. Accessed 9 September 2026.

[27] Wolfram Research, *ProductLog*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/ProductLog.html. Accessed 9 September 2026.

[28] Wolfram Research, *AsymptoticIntegrate*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/AsymptoticIntegrate.html. Accessed 9 September 2026.

[29] Wolfram Research, *AsymptoticDSolveValue*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/AsymptoticDSolveValue.html. Accessed 9 September 2026.

[30] Wolfram Research, *Interval*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/Interval.html. Accessed 9 September 2026.

[31] NIST Digital Library of Mathematical Functions, Section 25.2, Definition and Expansions of the Riemann Zeta Function. https://dlmf.nist.gov/25.2. Defining series and its fixed derivatives; accessed 9 September 2026.

[32] NIST Digital Library of Mathematical Functions, Section 25.14, Lerch's Transcendent. https://dlmf.nist.gov/25.14. Defining series; accessed 9 September 2026.